

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчет

по лабораторной работе №3

по дисциплине «Практика И Реализация Баз Данных»

Автор: Конанчук Иван Алексеевич

Факультет: ФПИИ

Группа: К3240

ИТМО

ЦЕЛЬ РАБОТЫ

Цель работы - создать и реализовать базу данных в PostgreSQL через pgAdmin по модели из ЛР 2, задать ограничения целостности, заполнить таблицы тестовыми данными, сгенерировать ERD, а потом сделать резервное копирование и проверить восстановление базы.

ПРАКТИЧЕСКОЕ ЗАДАНИЕ

- создать базу данных `azs_sales_db` в pgAdmin;
- создать схему `azs`;
- создать таблицы по модели предметной области "Анализ продаж автомобильного топлива в сети АЗС";
- задать ограничения Primary Key, Foreign Key, Unique и Check;
- заполнить таблицы рабочими данными;
- сгенерировать логическую схему базы данных в ERD;
- создать резервные копии в форматах Custom и Plain;
- восстановить базу данных из резервной копии и проверить результат.

ВЫПОЛНЕНИЕ

Наименование БД, схема и вариант

Наименование БД	<code>azs_sales_db</code>
Схема	<code>azs</code>
Вариант	Анализ продаж автомобильного топлива в сети АЗС

Сначала в pgAdmin была создана база данных `azs_sales_db`, после этого внутри нее была создана отдельная схема `azs`. Дальше вся работа велась уже внутри этой схемы.

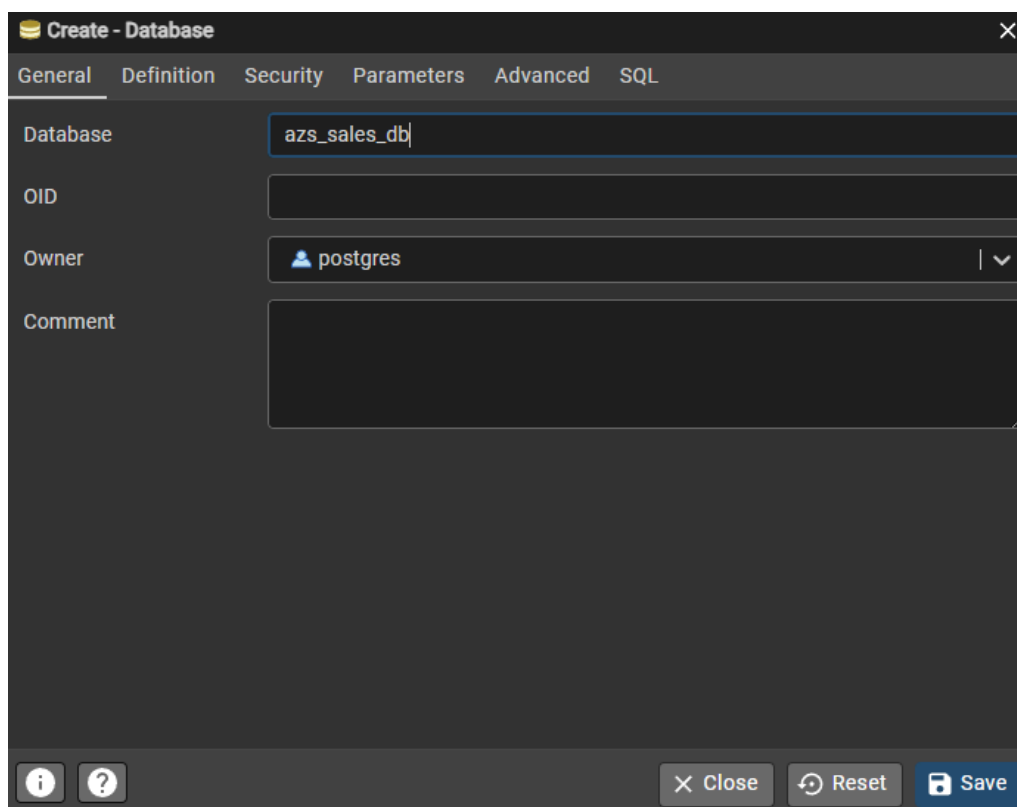


Рисунок 1 - Создание базы данных *azs_sales_db*

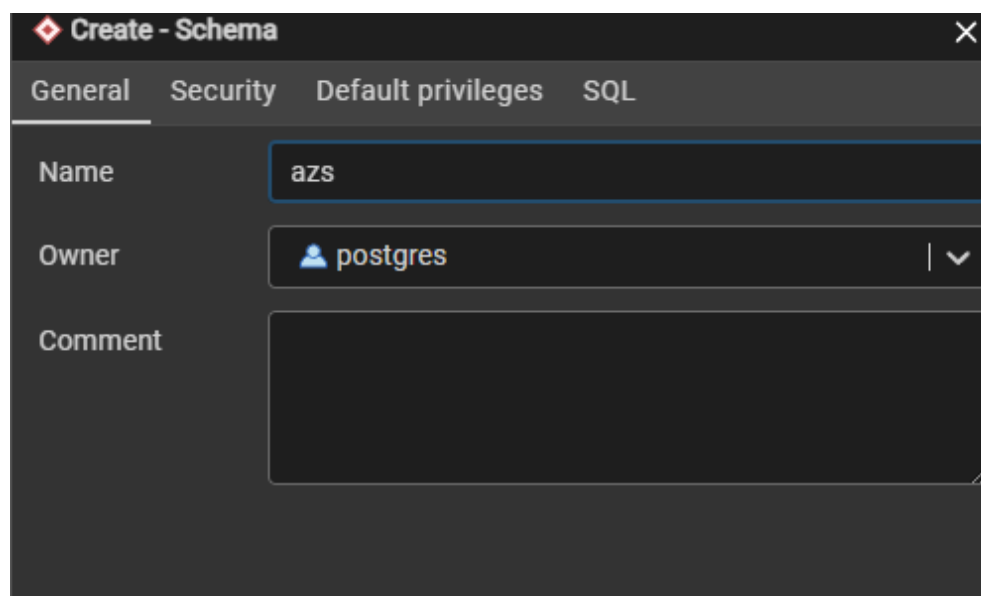
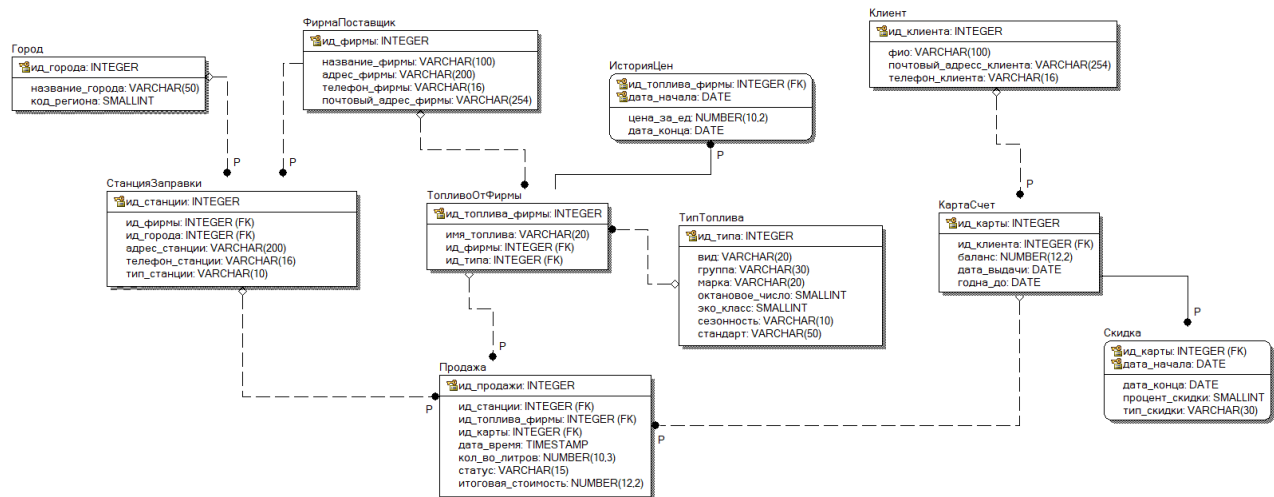


Рисунок 2 - Создание схемы *azs*

Схема инфологической модели БД ЛР 2 (IDEF1X)

Ниже в итоговой версии отчета будет вставлена инфологическая модель из ЛР 2. Фото в лучшем качестве по [ссылке](#)



Создание таблиц и ограничений

Первую таблицу city я создал вручную через интерфейс pgAdmin, чтобы показать работу с GUI. Остальные таблицы уже было удобнее и быстрее создавать через Query Tool на SQL.

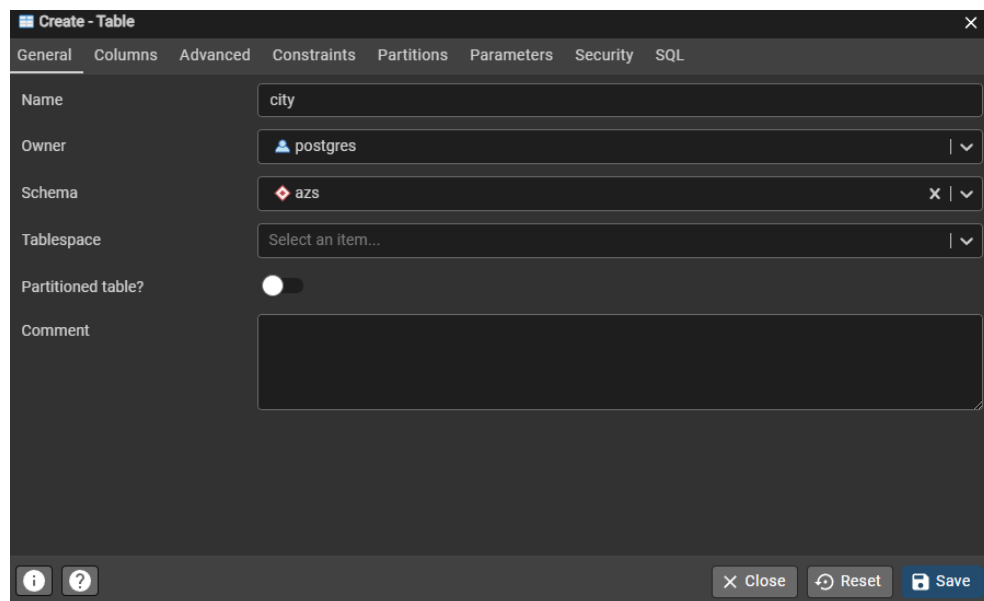


Рисунок 3 - Создание таблицы city через интерфейс pgAdmin

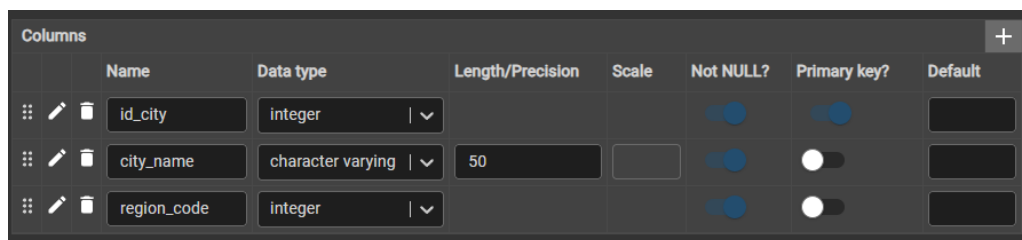


Рисунок 4 - Заполнение столбцов таблицы city

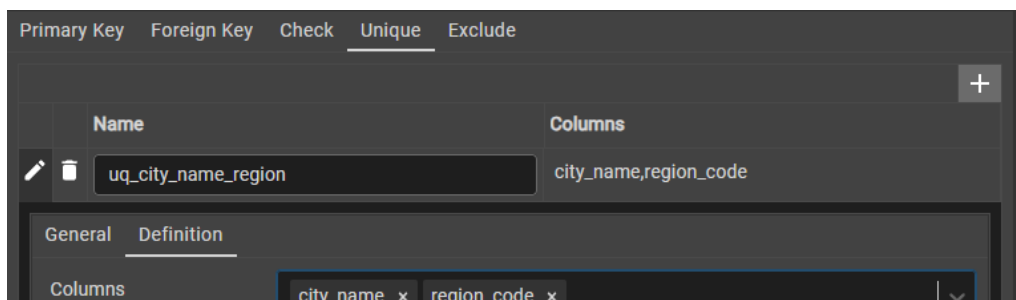


Рисунок 5 - Задание ограничения *UNIQUE* для таблицы *city*

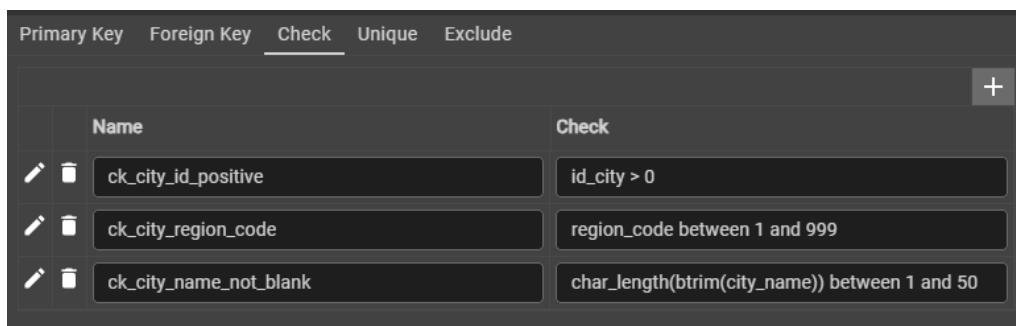


Рисунок 6 - Задание *CHECK*-ограничений для таблицы *city*

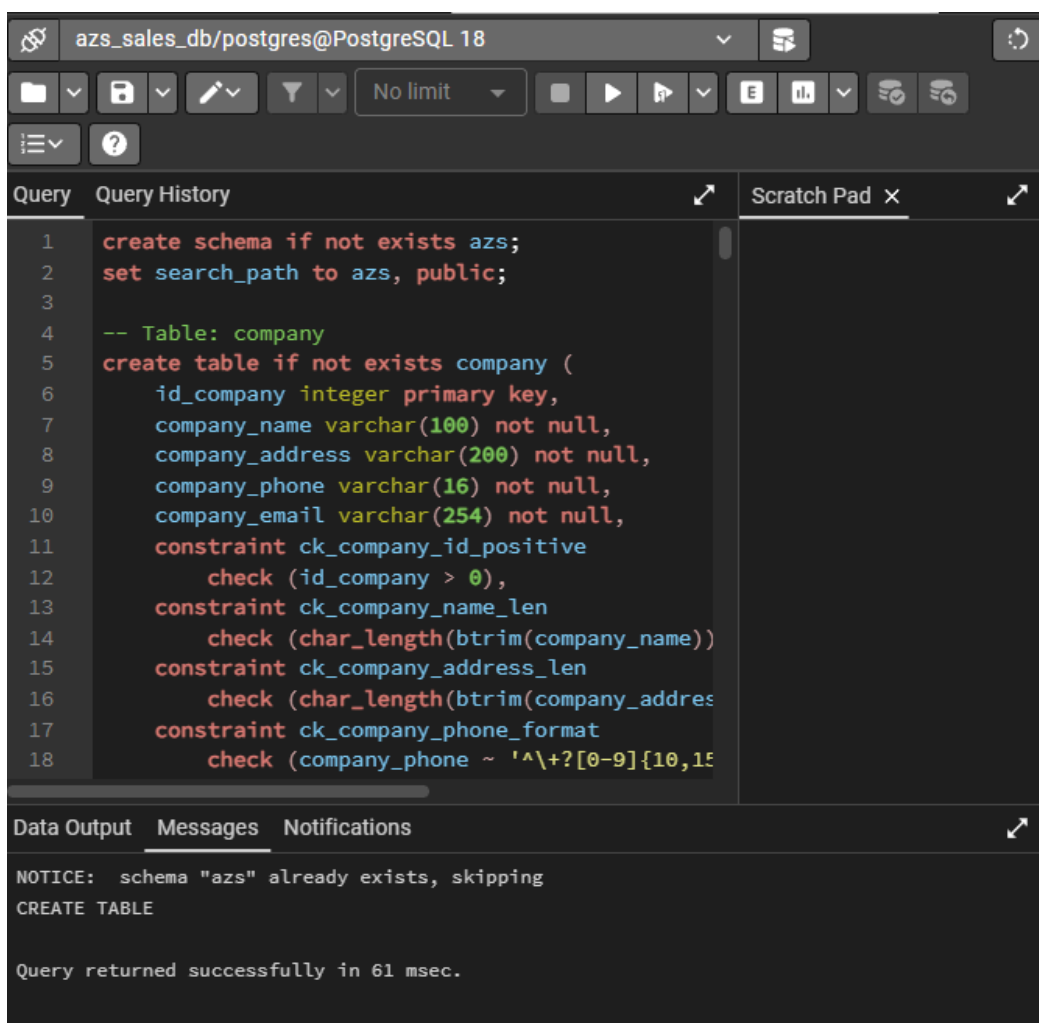
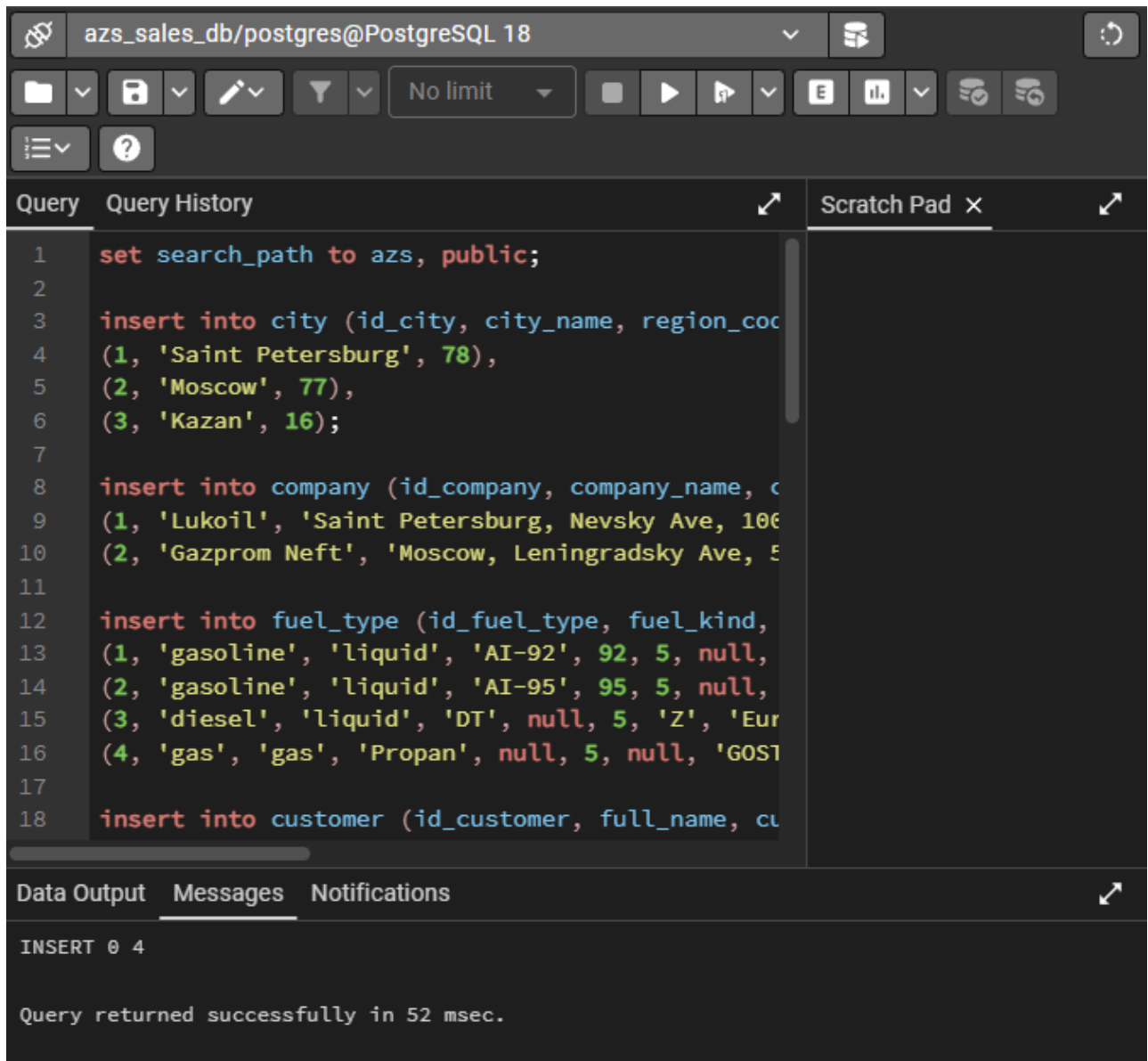


Рисунок 7 - Создание остальных таблиц через *Query Tool*

Заполнение таблиц и проверка данных

После создания структуры база была заполнена тестовыми данными через INSERT-запросы. Затем через SELECT была проверена корректность загрузки данных и связей между таблицами.



The screenshot shows a PostgreSQL client interface with the following components:

- Top Bar:** Connection name 'azs_sales_db/postgres@PostgreSQL 18' and various tool icons.
- Query Editor:** Contains the following SQL queries:

```
1 set search_path to azs, public;
2
3 insert into city (id_city, city_name, region_coc
4 (1, 'Saint Petersburg', 78),
5 (2, 'Moscow', 77),
6 (3, 'Kazan', 16);
7
8 insert into company (id_company, company_name, c
9 (1, 'Lukoil', 'Saint Petersburg, Nevsky Ave, 100
10 (2, 'Gazprom Neft', 'Moscow, Leningradsky Ave, 5
11
12 insert into fuel_type (id_fuel_type, fuel_kind,
13 (1, 'gasoline', 'liquid', 'AI-92', 92, 5, null,
14 (2, 'gasoline', 'liquid', 'AI-95', 95, 5, null,
15 (3, 'diesel', 'liquid', 'DT', null, 5, 'Z', 'Eur
16 (4, 'gas', 'gas', 'Propan', null, 5, null, 'GOST
17
18 insert into customer (id_customer, full_name, cu
```
- Query History:** Empty.
- Scratch Pad:** Empty.
- Data Output:** Shows the result of the last query: 'INSERT 0 4' and 'Query returned successfully in 52 msec.'
- Messages:** Empty.
- Notifications:** Empty.

Рисунок 8 - Заполнение таблиц тестовыми данными

The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is to 'azs_sales_db/postgres@PostgreSQL 18'. Below the toolbar, the 'Query' tab is active, displaying a SQL script with 12 lines. The script sets the search path and performs SELECT queries on various tables. The 'Data Output' tab is also visible, showing the results of the last query in a table format with 9 columns and 4 rows.

```

1  set search_path to azs, public;
2
3  select * from city;
4  select * from company;
5  select * from fuel_type;
6  select * from customer;
7  select * from station;
8  select * from company_fuel;
9  select * from card_account;
10 select * from price_history;
11 select * from discount;
12 select * from sale;

```

	id_sale [PK] integer	id_card integer	id_station integer	id_company_fuel integer	sale_datetime timestamp without time zone	liters numeric (10,3)	total_amount numeric (12,2)	status character varying (15)
1	1	1	1	1	2026-03-05 10:15:00	30.500	1677.65	completed
2	2	2	2	2	2026-03-06 12:40:00	20.000	1142.04	completed
3	3	3	4	4	2026-03-07 09:20:00	35.000	2339.16	completed
4	4	1	3	3	2026-03-08 18:05:00	15.250	915.61	completed

Рисунок 9 - Проверка данных через SELECT-запросы

Схема ERD

После создания таблиц и внешних ключей была сгенерирована логическая схема базы данных в ERD. На схеме видно, что связи между таблицами сформированы корректно, а все основные сущности и ассоциативная сущность sale связаны через внешние ключи.

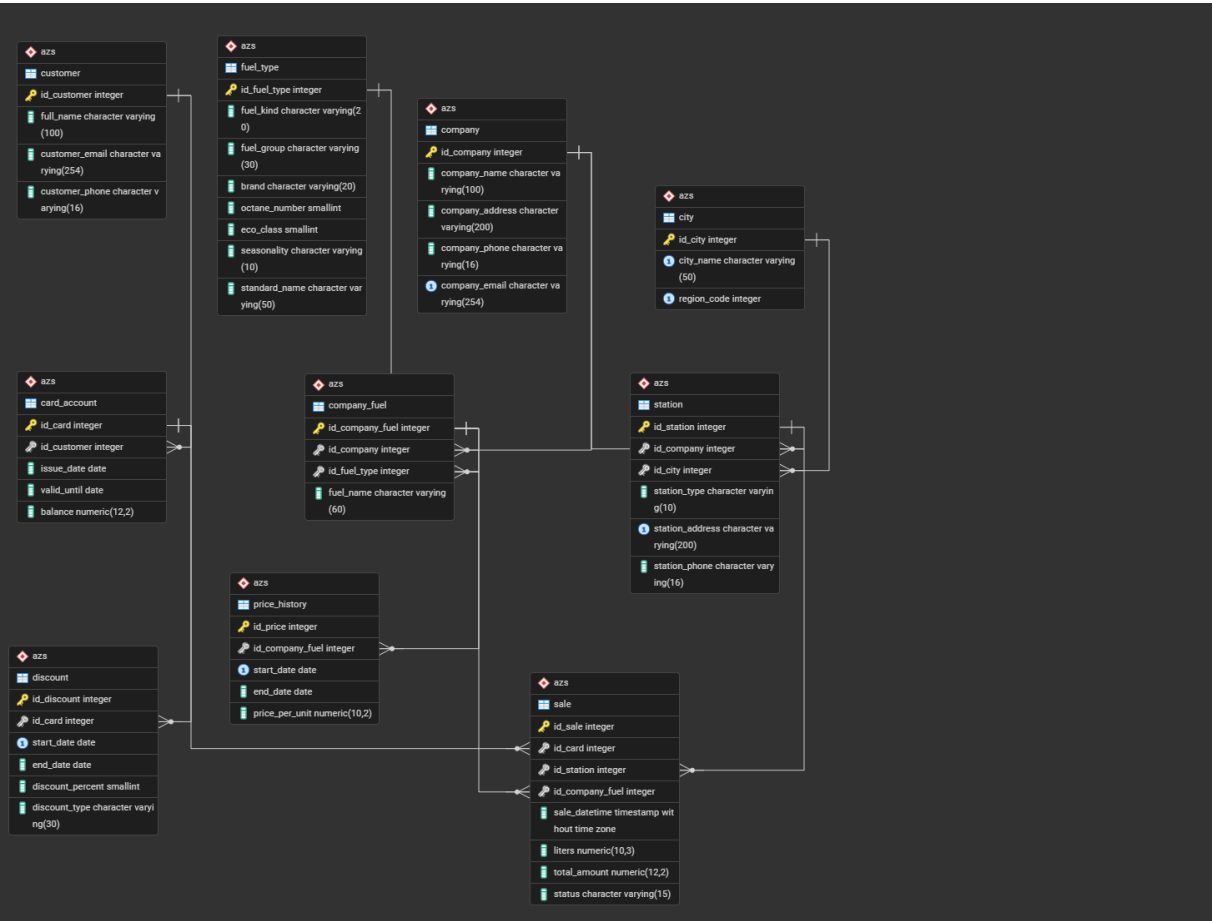


Рисунок 10 - Логическая модель базы данных ERD

В лучшем качестве по [ссылке](#)

Анализ составных ключей и замена их суррогатными простыми ключами

В инфологической модели составные первичные ключи были у сущностей ИсторияЦен и Скидка. В реализации базы данных эти составные ключи были заменены на суррогатные простые ключи, чтобы упростить физическую структуру таблиц, внешние связи и дальнейшую работу с данными.

Сущность	Составной ключ в ИЛМ	Суррогатный ключ в реализации	Как сохранена бизнес-уникальность
price_history	(id_company_fuel, start_date)	id_price	UNIQUE (id_company_fuel, start date)
discount	(id_card, start_date)	id_discount	UNIQUE (id_card, start date)

Резервное копирование и восстановление БД

После заполнения базы были созданы две резервные копии: Custom backup и Plain backup. Затем была создана пустая тестовая база и выполнено восстановление из backup-файла. По итогу схема azs и все 10 таблиц восстановились корректно.

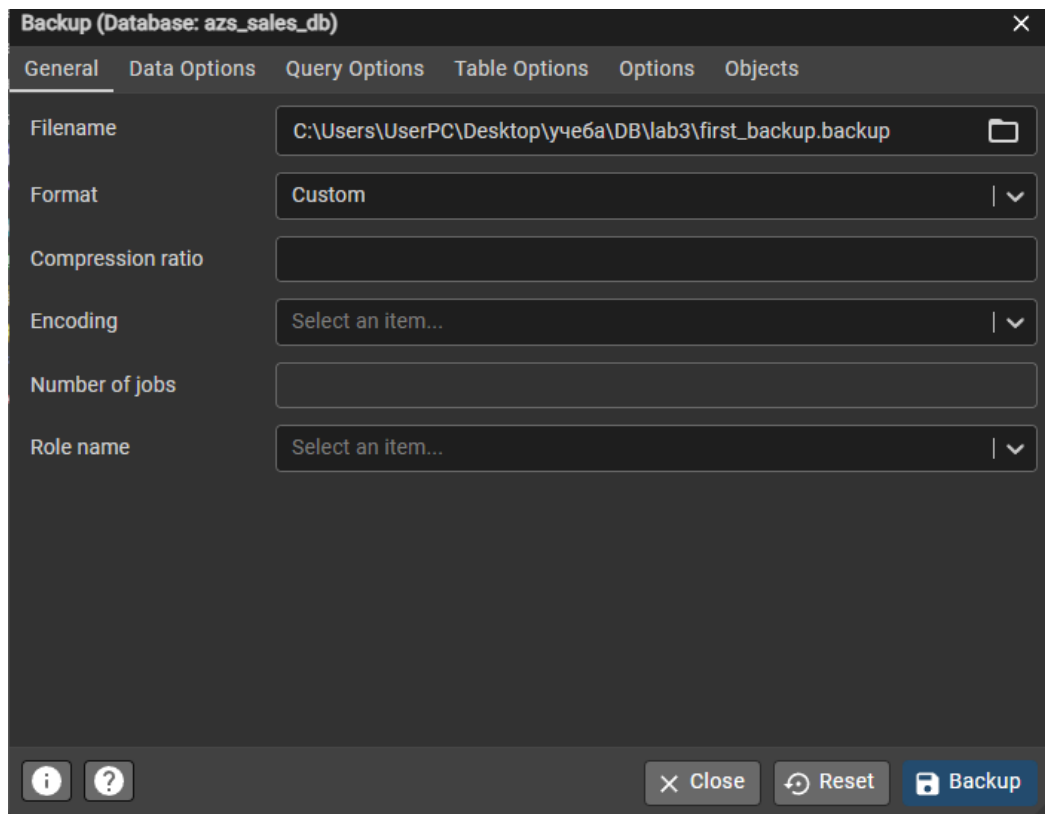


Рисунок 11 - Создание резервной копии в формате Custom

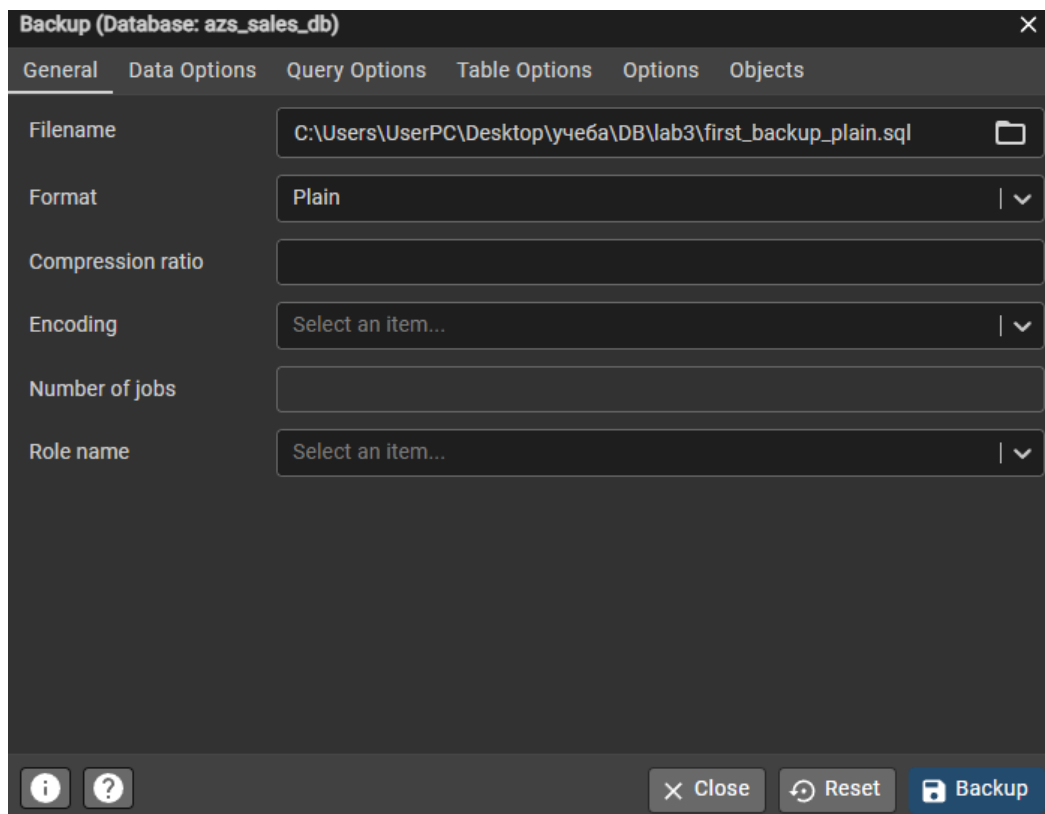


Рисунок 12 - Создание резервной копии в формате Plain

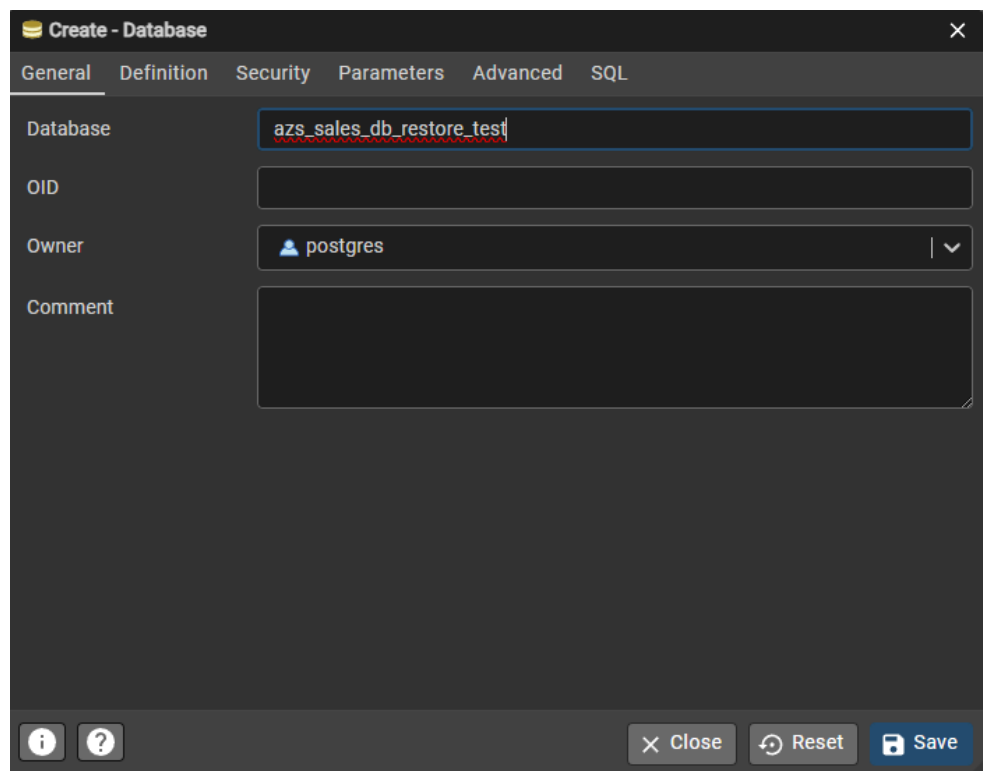


Рисунок 13 - Создание пустой базы данных для проверки восстановления

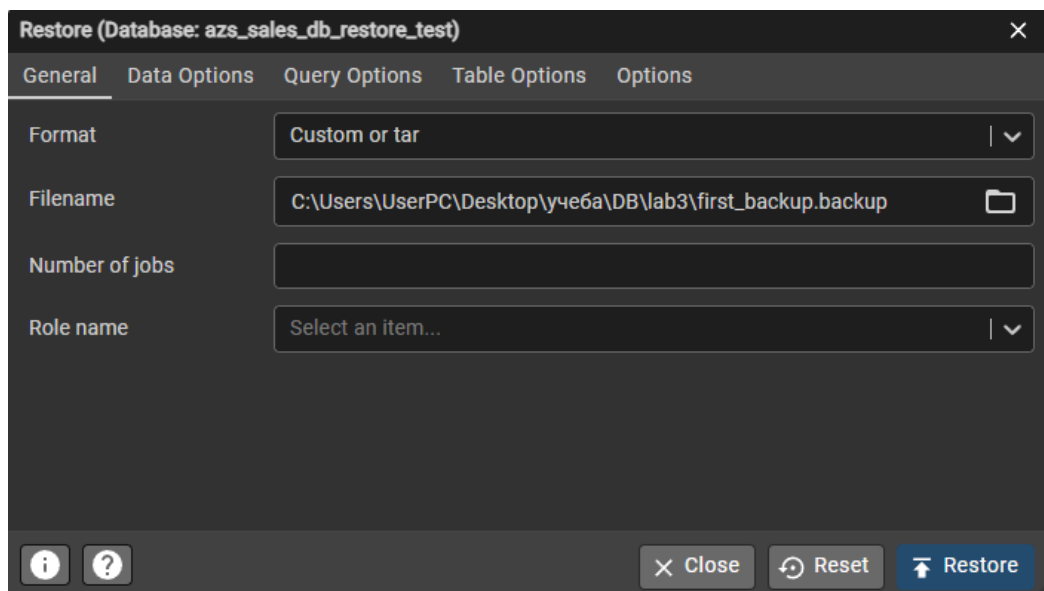


Рисунок 14 - Восстановление базы данных из файла backup

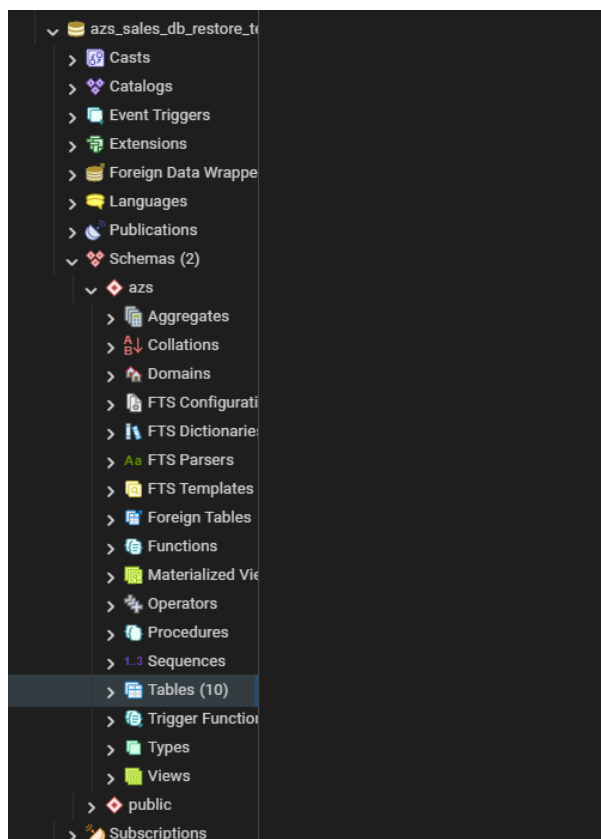


Рисунок 15 - Проверка восстановленной базы: схема azs и 10 таблиц на месте

DUMP, СОДЕРЖАЩИЙ СКРИПТЫ РАБОТЫ С БД

Ниже приведены основные SQL-скрипты. Создание базы данных, схемы, таблиц, ограничений, вставка тестовых данных и проверка.

```
-- Create database
CREATE DATABASE azs_sales_db;

-- Далее выполнение происходит внутри БД azs_sales_db

create schema if not exists azs;
set search_path to azs, public;

-- Table: company
create table if not exists company (
    id_company integer primary key,
    company_name varchar(100) not null,
    company_address varchar(200) not null,
    company_phone varchar(16) not null,
    company_email varchar(254) not null,
    constraint ck_company_id_positive
        check (id_company > 0),
    constraint ck_company_name_len
        check (char_length(btrim(company_name)) between 1 and 100),
    constraint ck_company_address_len
        check (char_length(btrim(company_address)) between 1 and 200),
    constraint ck_company_phone_format
        check (company_phone ~ '^\\+?[0-9]{10,15}$'),
    constraint ck_company_email_format
        check (company_email ~ '^([@\\s]+@[@\\s]+\\.([@\\s]+$))'),
    constraint uq_company_email
        unique (company_email)
);

-- Table: fuel_type
create table if not exists fuel_type (
    id_fuel_type integer primary key,
    fuel_kind varchar(20) not null,
    fuel_group varchar(30),
    brand varchar(20) not null,
    octane_number smallint,
    eco_class smallint not null,
    seasonality varchar(10),
    standard_name varchar(50),
    constraint ck_fuel_type_id_positive
        check (id_fuel_type > 0),
    constraint ck_fuel_type_kind
        check (fuel_kind in ('gasoline', 'diesel', 'gas')),
    constraint ck_fuel_type_group_len
        check (fuel_group is null or char_length(btrim(fuel_group)) <= 30),
    constraint ck_fuel_type_brand_len
```

```

        check (char_length(btrim(brand)) between 1 and 20),
constraint ck_fuel_type_octane
        check (octane_number is null or octane_number between 40 and 120),
constraint ck_fuel_type_eco_class
        check (eco_class between 0 and 6),
constraint ck_fuel_type_seasonality
        check (seasonality is null or seasonality in ('L', 'E', 'Z', 'A', 'VS')),
constraint ck_fuel_type_standard_len
        check (standard_name is null or char_length(btrim(standard_name)) <= 50)
);

-- Table: customer
create table if not exists customer (
    id_customer integer primary key,
    full_name varchar(100) not null,
    customer_email varchar(254) not null,
    customer_phone varchar(16) not null,
constraint ck_customer_id_positive
        check (id_customer > 0),
constraint ck_customer_full_name_len
        check (char_length(btrim(full_name)) between 1 and 100),
constraint ck_customer_full_name_words
        check (full_name ~ '^[A-Za-z]+([ -][A-Za-z]+)+$'),
constraint ck_customer_email_format
        check (customer_email ~ '^[^@\s]+@[^@\s]+\.[^@\s]+$'),
constraint ck_customer_phone_format
        check (customer_phone ~ '^\\+?[0-9]{10,15}$')
);

-- Table: station
create table if not exists station (
    id_station integer primary key,
    id_company integer not null,
    id_city integer not null,
    station_type varchar(10) not null,
    station_address varchar(200) not null,
    station_phone varchar(16) not null,
constraint ck_station_id_positive
        check (id_station > 0),
constraint ck_station_type
        check (station_type in ('AZS', 'AZGS')),
constraint ck_station_address_len
        check (char_length(btrim(station_address)) between 1 and 200),
constraint ck_station_phone_format
        check (station_phone ~ '^\\+?[0-9]{10,15}$'),
constraint uq_station_company_address
        unique (id_company, station_address),
constraint fk_station_company
        foreign key (id_company)
        references company (id_company)
        on update cascade

```

```

        on delete restrict,
constraint fk_station_city
    foreign key (id_city)
    references city (id_city)
    on update cascade
    on delete restrict
);

-- Table: company_fuel
create table if not exists company_fuel (
    id_company_fuel integer primary key,
    id_company integer not null,
    id_fuel_type integer not null,
    fuel_name varchar(60) not null,
    constraint ck_company_fuel_id_positive
        check (id_company_fuel > 0),
    constraint ck_company_fuel_name_len
        check (char_length(btrim(fuel_name)) between 1 and 60),
    constraint uq_company_fuel_company_type
        unique (id_company, id_fuel_type),
    constraint fk_company_fuel_company
        foreign key (id_company)
        references company (id_company)
        on update cascade
        on delete restrict,
    constraint fk_company_fuel_fuel_type
        foreign key (id_fuel_type)
        references fuel_type (id_fuel_type)
        on update cascade
        on delete restrict
);

-- Table: card_account
create table if not exists card_account (
    id_card integer primary key,
    id_customer integer not null,
    issue_date date not null,
    valid_until date not null,
    balance numeric(12,2) not null,
    constraint ck_card_account_id_positive
        check (id_card > 0),
    constraint ck_card_account_issue_date
        check (issue_date <= current_date),
    constraint ck_card_account_valid_until
        check (valid_until >= issue_date),
    constraint ck_card_account_balance
        check (balance >= 0),
    constraint fk_card_account_customer
        foreign key (id_customer)
        references customer (id_customer)
        on update cascade

```

```

        on delete restrict
    );

-- Table: price_history
create table if not exists price_history (
    id_price integer primary key,
    id_company_fuel integer not null,
    start_date date not null,
    end_date date,
    price_per_unit numeric(10,2) not null,
    constraint ck_price_history_id_positive
        check (id_price > 0),
    constraint ck_price_history_dates
        check (end_date is null or end_date >= start_date),
    constraint ck_price_history_price
        check (price_per_unit > 0),
    constraint uq_price_history_business
        unique (id_company_fuel, start_date),
    constraint fk_price_history_company_fuel
        foreign key (id_company_fuel)
        references company_fuel (id_company_fuel)
        on update cascade
        on delete cascade
);

-- Table: discount
create table if not exists discount (
    id_discount integer primary key,
    id_card integer not null,
    start_date date not null,
    end_date date,
    discount_percent smallint not null,
    discount_type varchar(30) not null,
    constraint ck_discount_id_positive
        check (id_discount > 0),
    constraint ck_discount_dates
        check (end_date is null or end_date >= start_date),
    constraint ck_discount_percent
        check (discount_percent between 0 and 100),
    constraint ck_discount_type_len
        check (char_length(btrim(discount_type)) between 1 and 30),
    constraint uq_discount_business
        unique (id_card, start_date),
    constraint fk_discount_card
        foreign key (id_card)
        references card_account (id_card)
        on update cascade
        on delete cascade
);

-- Table: sale

```

```

create table if not exists sale (
    id_sale integer primary key,
    id_card integer not null,
    id_station integer not null,
    id_company_fuel integer not null,
    sale_datetime timestamp not null,
    liters numeric(10,3) not null,
    total_amount numeric(12,2) not null,
    status varchar(15) not null,
    constraint ck_sale_id_positive
        check (id_sale > 0),
    constraint ck_sale_datetime
        check (sale_datetime <= now()),
    constraint ck_sale_liters
        check (liters > 0 and liters <= 500),
    constraint ck_sale_total_amount
        check (total_amount >= 0),
    constraint ck_sale_status
        check (status in ('completed', 'cancelled', 'processing')),
    constraint fk_sale_card
        foreign key (id_card)
        references card_account (id_card)
        on update cascade
        on delete restrict,
    constraint fk_sale_station
        foreign key (id_station)
        references station (id_station)
        on update cascade
        on delete restrict,
    constraint fk_sale_company_fuel
        foreign key (id_company_fuel)
        references company_fuel (id_company_fuel)
        on update cascade
        on delete restrict
);

-- fill_data.sql
set search_path to azs, public;

insert into city (id_city, city_name, region_code) values
(1, 'Saint Petersburg', 78),
(2, 'Moscow', 77),
(3, 'Kazan', 16);

insert into company (id_company, company_name, company_address, company_phone,
company_email) values
(1, 'Lukoil', 'Saint Petersburg, Nevsky Ave, 100', '+78120000001', 'office@lukoil.test'),
(2, 'Gazprom Neft', 'Moscow, Leningradsky Ave, 50', '+74950000002', 'info@gpn.test');

insert into fuel_type (id_fuel_type, fuel_kind, fuel_group, brand, octane_number, eco_class,
seasonality, standard_name) values

```



```

(1, 'gasoline', 'liquid', 'AI-92', 92, 5, null, 'Euro-5'),
(2, 'gasoline', 'liquid', 'AI-95', 95, 5, null, 'Euro-5'),
(3, 'diesel', 'liquid', 'DT', null, 5, 'Z', 'Euro-5'),
(4, 'gas', 'gas', 'Propan', null, 5, null, 'GOST');

insert into customer (id_customer, full_name, customer_email, customer_phone) values
(1, 'Ivan Ivanov', 'ivanov@test.ru', '+79000000001'),
(2, 'Petr Petrov', 'petrov@test.ru', '+79000000002'),
(3, 'Anna Sidorova', 'sidorova@test.ru', '+79000000003');

insert into station (id_station, id_company, id_city, station_type, station_address,
station_phone) values
(1, 1, 1, 'AZS', 'Saint Petersburg, Nevsky Ave, 10', '+78125550001'),
(2, 1, 2, 'AZS', 'Moscow, Leningradsky Ave, 15', '+74955550002'),
(3, 2, 1, 'AZS', 'Saint Petersburg, Moskovsky Ave, 25', '+78125550003'),
(4, 2, 3, 'AZGS', 'Kazan, Pobedy Ave, 40', '+78435550004');

insert into company_fuel (id_company_fuel, id_company, id_fuel_type, fuel_name) values
(1, 1, 1, 'AI-92 Lukoil'),
(2, 1, 2, 'AI-95 Lukoil'),
(3, 2, 2, 'AI-95 G-Drive'),
(4, 2, 3, 'DT Gazprom Neft');

insert into card_account (id_card, id_customer, issue_date, valid_until, balance) values
(1, 1, '2025-01-15', '2028-01-15', 5000.00),
(2, 2, '2025-03-10', '2028-03-10', 3200.00),
(3, 3, '2025-05-20', '2028-05-20', 1500.00);

insert into price_history (id_price, id_company_fuel, start_date, end_date, price_per_unit)
values
(1, 1, '2026-01-01', '2026-02-28', 56.90),
(2, 1, '2026-03-01', null, 57.90),
(3, 2, '2026-03-01', null, 61.40),
(4, 3, '2026-03-01', null, 63.20),
(5, 4, '2026-03-01', null, 68.90);

insert into discount (id_discount, id_card, start_date, end_date, discount_percent,
discount_type) values
(1, 1, '2026-01-01', null, 5, 'permanent'),
(2, 2, '2026-02-01', null, 7, 'personal'),
(3, 3, '2026-03-01', '2026-12-31', 3, 'promo');

insert into sale (id_sale, id_card, id_station, id_company_fuel, sale_datetime, liters,
total_amount, status) values
(1, 1, 1, 1, '2026-03-05 10:15:00', 30.500, 1677.65, 'completed'),
(2, 2, 2, 2, '2026-03-06 12:40:00', 20.000, 1142.04, 'completed'),
(3, 3, 4, 4, '2026-03-07 09:20:00', 35.000, 2339.16, 'completed'),
(4, 1, 3, 3, '2026-03-08 18:05:00', 15.250, 915.61, 'completed');

-- test.sql
set search_path to azs, public;

```

```
select * from city;
select * from company;
select * from fuel_type;
select * from customer;
select * from station;
select * from company_fuel;
select * from card_account;
select * from price_history;
select * from discount;
select * from sale;
```

ВЫВОДЫ

В ходе лабораторной работы была создана и реализована база данных azs_sales_db в PostgreSQL. Для таблиц были заданы первичные и внешние ключи, ограничения UNIQUE и CHECK, после чего база была заполнена тестовыми данными. Дополнительно была сгенерирована ERD-схема, создана резервная копия в форматах Custom и Plain и проверено восстановление базы в отдельную тестовую БД. В результате структура БД, связи между сущностями и данные были реализованы корректно.